

Проектирование интерфейсов

1. Общие сведения

Параметр	Значение
Наименование проекта	Шашки по сети
Этап жизненного цикла	Проектирование интерфейсов
Технологии	C#, .NET, Windows Forms, TCP/IP

4. Общая концепция пользовательского интерфейса

Пользовательский интерфейс должен быть простым, понятным и реализуемым стандартными средствами Windows Forms. Интерфейс не должен требовать сложной стилизации, нестандартных анимаций или дополнительных библиотек.

Основная задача интерфейса - дать пользователю возможность создать сетевую игру, подключиться ко второму игроку, видеть игровое поле, понимать текущий ход и получать сообщения об ошибках или состоянии соединения.

4.1. Общие требования к интерфейсу

- интерфейс должен быть выполнен на русском языке;
- основные действия должны быть доступны из главного окна приложения;
- игровое поле должно занимать центральную часть окна и быть хорошо различимым;
- элементы управления соединением должны быть сгруппированы отдельно от игрового поля;
- сообщения об ошибках и ходе партии должны быть понятными для пользователя;
- недоступные действия должны блокироваться;
- приложение не должно завершаться аварийно при ошибках подключения, неверном ходе или потере соединения.

4.2. Основные блоки главного окна

Блок интерфейса	Состав	Назначение
Блок подключения	Поле IP-адреса, поле порта, кнопка «Создать сервер», кнопка «Подключиться», кнопка «Отключиться»	Позволяет одному игроку создать сервер, а второму подключиться к нему по локальной сети.

Блок интерфейса	Состав	Назначение
Игровое поле	Поле 8x8, клетки двух цветов, шашки двух игроков, выделение выбранной шашки	Отображает состояние партии и принимает действия пользователя по выбору шашки и клетки назначения.
Панель состояния	Текущий игрок, роль пользователя, состояние соединения, состояние партии	Показывает, чей сейчас ход, подключен ли второй игрок и в каком состоянии находится игра.
Область сообщений	Список или текстовое поле для служебных сообщений	Выводит сообщения о подключении, ходе соперника, неверном ходе, победе и потере соединения.
Блок управления партией	Кнопка «Новая игра» или «Сброс», при необходимости кнопка выхода из партии	Позволяет начать новую партию или сбросить состояние после завершения игры.

4.3. Структура окна

Для реализации в Windows Forms будет одно главное окно MainForm. В верхней части окна размещается блок подключения, в центральной части игровое поле 8 на 8, сбоку или снизу панель состояния и область сообщений.

5. Требования к игровому полю

Игровое поле является главным визуальным элементом приложения. Оно должно отображать доску 8x8, шашки двух игроков и результат каждого корректного хода.

- поле состоит из 8 строк и 8 столбцов;
- клетки должны визуально чередоваться по цвету;
- шашки двух игроков должны отличаться цветом;
- простая шашка и дамка должны иметь различимое обозначение;
- выбранная шашка должна выделяться изменением цвета клетки;
- если ход выполнен корректно, поле должно сразу обновляться у обоих игроков;
- если ход некорректен, состояние поля не должно изменяться;
- во время хода соперника выбор своих шашек должен быть заблокирован;

6. Поведение интерфейса по состояниям приложения

Состояние приложения	Поведение интерфейса	Сообщение пользователю
Запуск приложения	Отображается главное окно, доступны элементы создания сервера и подключения к серверу. Игровое поле может быть пустым или находиться в стартовом состоянии.	Выберите режим: создать сервер или подключиться к игре.
Ожидание подключения	После нажатия «Создать сервер» блокируется повторный запуск сервера. Поле может отображаться, но ходы недоступны до подключения второго игрока.	Сервер запущен. Ожидание второго игрока.
Подключение клиента	После успешного подключения активируется игровое поле. Пользователю назначается цвет или роль в партии.	Подключение выполнено. Игра начинается.
Ход текущего игрока	Пользователь может выбрать свою шашку и клетку назначения. Выбранная шашка подсвечивается.	Ваш ход.
Ход соперника	Пользователь видит поле, но не может выполнить ход до получения сообщения от второго игрока.	Ожидание хода соперника.
Некорректный ход	Поле остается без изменений. Выбор может быть сброшен, чтобы пользователь выбрал ход заново.	Ход невозможен: нарушены правила игры.
Окончание партии	Игровое поле блокируется для новых ходов. Показывается победитель. При нажатии кнопки «Новая игра» пользователь может начать новую партию.	Партия завершена. Имя победителя
Потеря соединения	Игровые действия блокируются. Пользователь получает сообщение о проблеме соединения.	Соединение со вторым игроком потеряно.

7. Основные пользовательские сценарии

7.1. Создание игры

- пользователь запускает приложение;
- указывает порт или оставляет значение по умолчанию;
- нажимает кнопку «Создать сервер»;
- приложение запускает TCP-сервер и переходит в состояние ожидания подключения;

- после подключения второго игрока приложение отображает начальную расстановку шашек и разрешает ход первому игроку согласно правилам партии.

7.2. Подключение к игре

- пользователь запускает второй экземпляр приложения;
- вводит IP-адрес и порт сервера;
- нажимает кнопку «Подключиться»;
- при успешном подключении приложение получает статус клиента и отображает игровое поле;
- при ошибке подключения приложение выводит понятное сообщение и позволяет повторить ввод данных.

7.3. Выполнение хода

- игрок выбирает свою шашку на игровом поле;
- интерфейс сохраняет выбранную клетку и визуально выделяет ее;
- игрок выбирает клетку назначения;
- MainForm формирует объект Move и передает его в игровую логику;
- игровая логика проверяет корректность хода через MoveValidator;
- при успешной проверке обновляется GameBoard, перерисовывается поле и ход передается второму игроку через NetworkMessage;
- при ошибке интерфейс показывает сообщение, а состояние доски не изменяется.

8. API взаимодействия между модулями

API взаимодействия строится вокруг принципа разделения ответственности. MainForm отвечает только за отображение и действия пользователя. GameLogic отвечает за правила игры и состояние партии. Network отвечает за передачу сообщений между экземплярами приложения. UI не должен самостоятельно проверять правила шашек и не должен напрямую изменять состояние доски без обращения к игровой логике.

8.1. Связь UI / WinForms с игровой логикой

Элемент API	Тип	Назначение
StartNewGame()	метод CheckersGame	Создает начальное состояние партии и расставляет шашки на доске.

Элемент API	Тип	Назначение
TryMakeMove(Move move)	метод CheckersGame	Проверяет и выполняет ход. Возвращает результат выполнения хода.
GetBoardSnapshot()	метод CheckersGame	Возвращает текущее состояние игрового поля для отрисовки в интерфейсе.
GetCurrentPlayer()	метод CheckersGame	Возвращает игрока, который должен выполнить следующий ход.
CheckWinner()	метод CheckersGame	Проверяет, завершена ли партия и кто является победителем.
BoardChanged	событие	Сообщает интерфейсу, что состояние доски изменилось и поле нужно перерисовать.
GameStateChanged	событие	Сообщает интерфейсу об изменении состояния партии.

8.2. Связь игровой логики с моделями данных

Модель	Основные поля	Назначение
Move	fromRow, fromCol, toRow, toCol	Описывает ход игрока от начальной клетки к конечной клетке.
CellPosition	row, col	Описывает координаты клетки игрового поля.
Piece	color, type	Описывает шашку, ее цвет и тип: простая шашка или дамка.
PlayerColor	White, Black	Задаёт цвет игрока.
GameState	WaitingForConnection, Playing, WhiteWon, BlackWon, ConnectionError, GameOver	Описывает текущее состояние партии.
MoveResult	success, message, capturedPiece, winner	Возвращает результат попытки выполнить ход и текст для отображения пользователю.

8.3. Связь UI / WinForms с сетевым модулем

Элемент API	Тип	Назначение
StartServer(int port)	метод TcpGameServer	Запускает сервер и переводит приложение в режим ожидания второго игрока.
Connect(string ipAddress, int port)	метод TcpGameClient	Подключает клиента к серверу по IP-адресу и порту.
SendMessage(NetworkMessage message)	метод TcpGameServer / TcpGameClient	Отправляет сетевое сообщение второму экземпляру приложения.
ReceiveMessage()	метод TcpGameServer / TcpGameClient	Получает сообщение из TCP-соединения и передает его для обработки.
Stop() / Disconnect()	метод	Закрывает соединение и освобождает сетевые ресурсы.
MessageReceived	событие	Вызывается при получении сетевого сообщения.
ConnectionLost	событие	Вызывается при потере соединения со вторым игроком.

8.4. Связь игровой логики с сетевым модулем

Игровая логика не должна напрямую работать с TcpListener, TcpClient, потоками или IP-адресами. После успешного локального хода MainForm получает результат от CheckersGame и формирует сетевое сообщение типа move. На втором экземпляре приложения сетевой модуль получает сообщение, MainForm преобразует его в Move и передает в CheckersGame для применения к локальному состоянию партии.

9. Формат сетевых сообщений

Будет использован простой текстовый формат JSON, передаваемый построчно.

Тип сообщения	Назначение	Основные данные
connect	Сообщение о подключении игрока.	type, playerColor
move	Передача выполненного хода второму игроку.	type, fromRow, fromCol, toRow, toCol
gameOver	Сообщение об окончании партии и победителе.	type, winner
error	Сообщение о сетевой ошибке или некорректном состоянии.	type, errorText
disconnect	Сообщение о завершении соединения.	type, reason

9.1. Пример сообщения о ходе

```
{ "type": "move", "fromRow": 5, "fromCol": 0, "toRow": 4, "toCol": 1 }
```

Конкретные названия классов и полей могут быть уточнены при реализации, но структура сообщений должна оставаться стабильной и понятной.

10. Последовательность взаимодействия модулей

Последовательность взаимодействия модулей должна соответствовать ранее выполненному системному и детальному проектированию: MainForm принимает действие пользователя, CheckersGame управляет партией, MoveValidator проверяет ход, GameBoard хранит состояние доски, а NetworkMessage передает подтвержденный ход второму игроку.

№	Действие	Участвующие модули
1	Игрок выбирает шашку и клетку назначения.	MainForm, UI
2	Интерфейс создает объект Move.	MainForm, Move
3	Ход передается в CheckersGame.TryMakeMove().	MainForm, CheckersGame
4	MoveValidator проверяет диагональность, владельца шашки, свободную клетку, взятие и очередность.	CheckersGame, MoveValidator, GameBoard
5	При успешной проверке обновляется состояние доски.	CheckersGame, GameBoard
6	Интерфейс перерисовывает игровое поле и обновляет индикатор текущего хода.	MainForm, UI
7	Сетевой модуль отправляет сообщение move второму игроку.	MainForm, NetworkMessage, TcpGameServer / TcpGameClient
8	Второй экземпляр принимает сообщение, применяет ход и обновляет свое поле.	Network, MainForm, CheckersGame

11. Требования к обработке ошибок в интерфейсе и API

- при неверном IP-адресе или порте пользователь должен получить сообщение о некорректных данных;
- при невозможности подключения к серверу приложение должно показать сообщение и оставить возможность повторного подключения;
- при потере соединения игровые действия должны быть заблокированы;

- при получении неизвестного типа сетевого сообщения приложение должно вывести служебную ошибку и не изменять состояние партии;
- при некорректном ходе CheckersGame должен возвращать результат с `success = false` и поясняющим текстом;
- исключения сетевого модуля не должны приводить к аварийному завершению MainForm;
- состояние GameBoard должно изменяться только после успешной проверки MoveValidator.

12. Связь с требованиями технического задания

Требование	Как учитывается в проектировании интерфейса и API
FR-03, FR-04	В интерфейсе предусмотрены элементы создания TCP-сервера и подключения по IP-адресу и порту.
FR-05, FR-06	Игровое поле 8x8 и шашки двух цветов являются центральным элементом MainForm.
FR-07, FR-08	Интерфейс поддерживает выбор шашки и выбор клетки назначения.
FR-09, FR-10, FR-11	Проверка хода выполняется через CheckersGame и MoveValidator, а не напрямую в UI.
FR-12, FR-13	После успешного хода формируется NetworkMessage типа move, поле обновляется у обоих игроков.
FR-14, FR-15	Панель состояния показывает текущего игрока, а окончание партии отображается сообщением о победителе.
FR-16	При потере соединения интерфейс показывает сообщение и блокирует игровые действия.
NFR-03	Интерфейс описан как простой, русскоязычный и понятный без дополнительного обучения.
NFR-04, NFR-05	API разделяет WinForms, игровую логику и сетевой код.
NFR-07, NFR-08	Ошибки ходов и соединения обрабатываются без аварийного завершения приложения.